

Communication 3 — 38 kHz Modulated IR

Tutorial Group: Communication

Goal

This tutorial extends the short IR communication link from *Communication_Short_IR_Communication* by adding a 38 kHz infrared carrier to the 1200-baud UART data stream. The first build uses an Arduino Mega 2560 transmitter and an ESP32 receiver.

- Understand the difference between the 1200-baud UART data rate and the 38 kHz carrier.
- Wire an IR LED between the Mega 2560's carrier output and UART TX output.
- Use a TSSP93038SS1ZA to detect the 38 kHz infrared signal.
- Verify the receiver before attempting full message communication.

What You Need

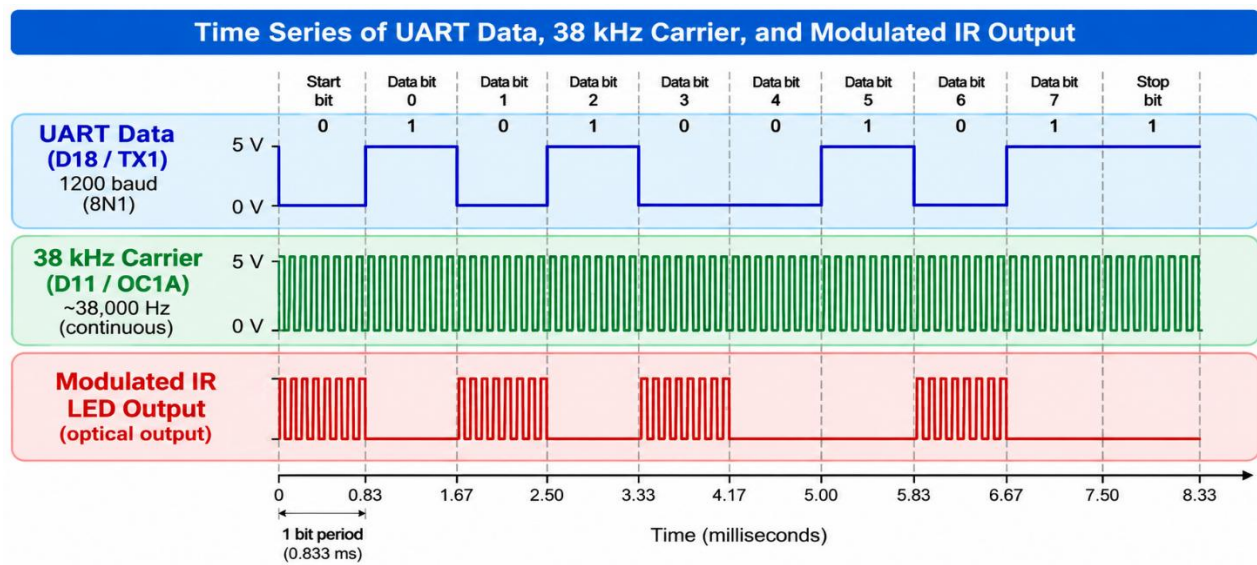
- Arduino Mega 2560
- ESP32 development board
- Two breadboards – one dedicated to the Arduino Mega 2560 and one dedicated to the ESP32, and jumper wires
- IR LED suitable for 38 kHz modulation
- 330 Ω resistor
- Vishay TSSP93038SS1ZA 38 kHz IR demodulator
- An oscilloscope or logic analyzer (recommended)

Part 1 — The two timing signals

In the previous tutorial, the detector only looked for presence of an IR signal. The problem with that approach is that everything that emits thermal radiation emits infrared light, including sources such as the sun, incandescent lights, candles, your soldering iron, your laptop, and even objects at ordinary temperatures.

One solution to that problem is to replace the steady IR beam with a much higher-frequency carrier that is transmitted only while the UART signal is LOW.

For this tutorial, we will continue to use the 1200-baud UART signal, but we will modulate the signal with a 38 kHz carrier frequency. UART normally idles HIGH, so no carrier is transmitted when the UART line is idle. When the UART signal goes LOW, the 38 kHz carrier is transmitted as a burst of infrared light. Any HIGH intervals, including the normal UART idle state, will continue to be OFF.



Part 2 — The Transmitter

The modulated UART stream is produced by connecting the IR LED to the UART output through the current-limiting resistor, and connecting the other side of the LED to the 38 kHz carrier output. When the UART output is LOW, the 38 kHz carrier controls the IR LED, producing bursts of infrared light at the carrier frequency. When the UART output is HIGH, the IR LED remains OFF.

With the Mega 2560, we can use one of the built-in hardware timers to generate the 38 kHz carrier wave. This allows the carrier frequency to be generated in the background without any additional software once the timer is started.

As in the previous tutorial, we will continue to use D18 / TX1 for the UART stream, but we will use Timer1 to generate the 38 kHz modulated carrier on OC1A.

Arduino Mega 2560 – 38 kHz Modulated IR Transmitter



MEGA2560 Connections

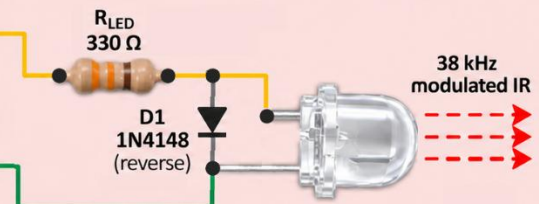
D18 / TX1

1200-baud UART
(from Mega)

D11 / OC1A

38 kHz carrier
(from Mega)

IR LED Transmitter Circuit (view from side, IR LED pointing to the right)



Important

- The reverse-protection diode (D1, 1N4148) is connected across the IR LED (anode to D18, cathode to D11).
- Keep the IR LED aligned and shielded (e.g., inside a paper tube).
- D11 generates the 38 kHz carrier and D18 (Serial1 TX) provides the 1200-baud UART data.
- The LED cathode (shorter lead) connects to D18 through the resistor.
- The LED anode (longer lead) connects to D11.

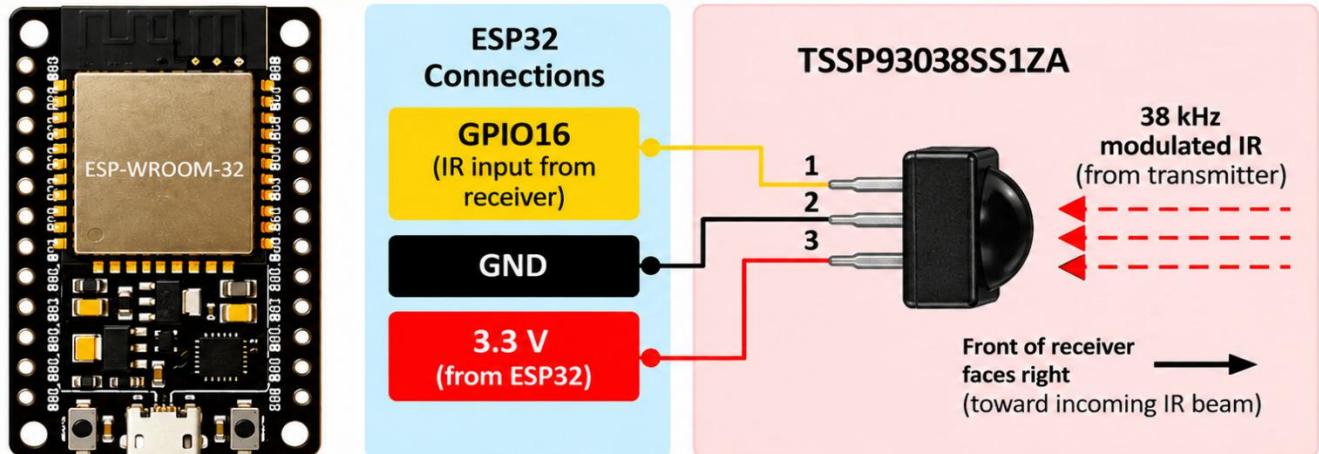
⚠ IMPORTANT: The resistor limits forward current through the IR LED; it does not automatically protect the IR LED from reverse voltage. Because the IR LED is connected between two push-pull digital outputs, the voltage across the IR LED can reverse during part of the carrier cycle. A reverse-protection diode (D1) must be connected across the IR LED, parallel to the LED, so it conducts during the reverse-voltage half-cycle. Connect the D1 diode cathode to the IR LED anode and the diode anode to the IR LED cathode. The resistor still limits the LED's forward current and now also limits the reverse current through the D1 diode.

Part 3 — The Receiver

The receiver uses a Vishay TSSP93038SS1ZA 38 kHz infrared demodulator. It detects the modulated 38 kHz optical signal and provides a digital output to the ESP32.

At 1200 baud, a single bit lasts approximately 833 μ s. When modulated at 38 kHz, this provides approximately 32 carrier cycles during each active bit. The TSSP93038SS1ZA datasheet recommends an input burst duration of at least 10 carrier cycles for optimal operation. At 1200 baud, the carrier burst is therefore substantially longer than the recommended minimum.

Receiver (ESP32) – 38 kHz Modulated IR Receiver



Important

- Pin 1 (top) is OUT and connects to ESP32 GPIO16 (IR input).
- Pin 2 (middle) is GND.
- Pin 3 (bottom) is VS and connects to 3.3 V from the ESP32.
- The front of the IR receiver faces to the right (toward the incoming IR LED beam).
- Keep the IR receiver aligned and shielded (e.g., inside a paper tube) to reduce ambient light.
- The IR receiver demodulates the 38 kHz signal and outputs a clean digital signal to GPIO16.
- Use the same UART data rate (1200 baud) on the transmitter and receiver.

Part 4 — Transmitter Carrier Test

If you have access to an oscilloscope or logic analyzer (or the DSO-TC3 mentioned in the *Getting Started*), you can verify the Mega 2560's 38 kHz carrier before attempting a complete message.

Mega 2560 38 kHz Carrier Test Code

```
// Communication_38kHz_Modulated_IR
// Arduino Mega 2560
// 38 kHz IR carrier test
//
// Timer1 generates the carrier on digital pin 11 (OC1A).

const int CARRIER_PIN = 11;
const int LED_RETURN_PIN = 18;

void setup()
{
  // Pin 11 is the Timer1 OC1A output.
  pinMode(CARRIER_PIN, OUTPUT);

  // Stop Timer1 while configuring it.
  TCCR1A = 0;
  TCCR1B = 0;

  // Start the timer at zero.
  TCNT1 = 0;

  // CTC compare value.
  //
  // Frequency = 16,000,000 / (2 * (OCR1A + 1))
  //
```

```

// With OCR1A = 209:
//
// Frequency = 16,000,000 / (2 * 210)
//           = 38,095 Hz
//
// This is approximately 38 kHz.
OCR1A = 209;

// CTC mode.
TCCR1B |= (1 << WGM12);

// Toggle OC1A (pin 11) every time Timer1 reaches OCR1A.
TCCR1A |= (1 << COM1A0);

// Start Timer1 with no prescaler.
TCCR1B |= (1 << CS10);

// Hold the LED return side LOW.
pinMode(LED_RETURN_PIN, OUTPUT);
digitalWrite(LED_RETURN_PIN, LOW);
}

void loop()
{
    // The hardware timer continuously generates
    // approximately 38 kHz on pin 11.
}

```

On the Arduino Mega 2560, Timer1 is configured to generate approximately 38 kHz on digital pin 11 (OC1A). With a 16 MHz clock and OCR1A = 209, the calculated frequency is $16,000,000 / (2 \times 210) = 38,095$ Hz.

Verify the actual waveform with an oscilloscope or logic analyzer.

Part 5 — Receiver Test

Before trying to receive UART characters, now verify that the TSSP93038SS1ZA is responding to the 38 kHz infrared signal from Part 4.

Receiver Test Code

```

// Communication_38kHz_Modulated_IR
// ESP32 - 38 kHz IR carrier detector

const int IR_RX_PIN = 16;

void setup()
{
    Serial.begin(115200);
    pinMode(IR_RX_PIN, INPUT);
}

void loop()
{
    Serial.println(digitalRead(IR_RX_PIN));
    delay(100);
}

```

Open the ESP32 Serial Monitor at 115200 baud. With no 38 kHz infrared signal present, the TSSP93038SS1ZA output should be **HIGH**. When the 38 kHz IR signal is detected, the TSSP93038SS1ZA output becomes **LOW** because its output is active-low. With the continuous 38 kHz carrier running, aim the light at the TSSP93038SS1ZA and observe the change in the Serial Monitor.

For this test, use a continuous 38 kHz IR signal rather than the modulated UART signal; the 100 ms polling interval is intentionally slow and may miss the short UART bursts.

Expected result: No detected IR signal → **HIGH** Detected 38 kHz IR signal → **LOW**

Part 6 — Communication Test: UART Data Modulates the 38 kHz Carrier

The previous tests verified the carrier and receiver separately. Now combine the UART code from the previous tutorial with the 38 kHz carrier code. Use the TSSP93038SS1ZA receiver with the ESP32 UART receiver code from the previous tutorial to receive the recovered 1200-baud UART data.

Because the carrier is transmitted during UART LOW intervals and the TSSP93038SS1ZA output is active-low, the receiver output follows the original UART logic.

Part 7 — Experiments

- Change the distance between transmitter and receiver.
- Change the alignment between the IR LED and TSSP93038SS1ZA.
- Test different room-lighting conditions.
- Try a flashlight near the receiver.
- Add paper-tube shielding.
- Compare the 38 kHz modulated link with an unmodulated IR link.
- Try sending data at slower and faster baud rates. Remember to make sure both devices operate at the same baud rate. At what baud rate does the communication no longer work?
- With your knowledge of the TSSP93038SS1ZA you learned in the previous parts, can you calculate a theoretical upper limit for the baud rate from the TSSP93038SS1ZA timing requirements? How does that compare with the highest baud rate at which the communication actually works?
- As an extension, investigate sending data both ways: from the Mega 2560 to the ESP32, and then back from the ESP32 to the Mega 2560.

What You Should Understand

This circuit contains two very different timing signals. The UART changes at 1200 baud, while the infrared carrier changes at approximately 38 kHz. Understanding how those two signals coexist is the central concept of this tutorial and could be very important to understanding this (or future) SoutheastCon Circuit Design competitions.

Before Moving On

You should be able to answer **yes** to all of the following:

- ☐ I understand why the IR signal is modulated at approximately 38 kHz.
- ☐ I can explain the difference between the information being transmitted and the 38 kHz carrier.
- ☐ I built the IR transmitter and receiver circuits.
- ☐ I uploaded and ran the transmitter test program.
- ☐ I observed the 38 kHz modulation on an oscilloscope or logic analyzer.
- ☐ I observed the recovered signal from the IR receiver.
- ☐ I understand how the IR receiver detects the modulated signal.

You do not need to memorize the circuit or code. The important thing is that you understand what is happening to the signal as it travels from the Arduino, through the IR transmitter and receiver, and into the ESP32.

If any of these points are unclear, go back to the corresponding section of this tutorial before moving on.